

5. CNN

- **INDEX**

- CNN의 등장
- CNN
- 다양한 CNN 모델

CNN의 등장

✓ 이미지 데이터의 특징

- 이미지는 픽셀들이 2차원 공간 구조를 가진 데이터 (이미지 추가)
- 컴퓨터는 이미지를 픽셀값의 집합으로 받아들임(이미지 추가)

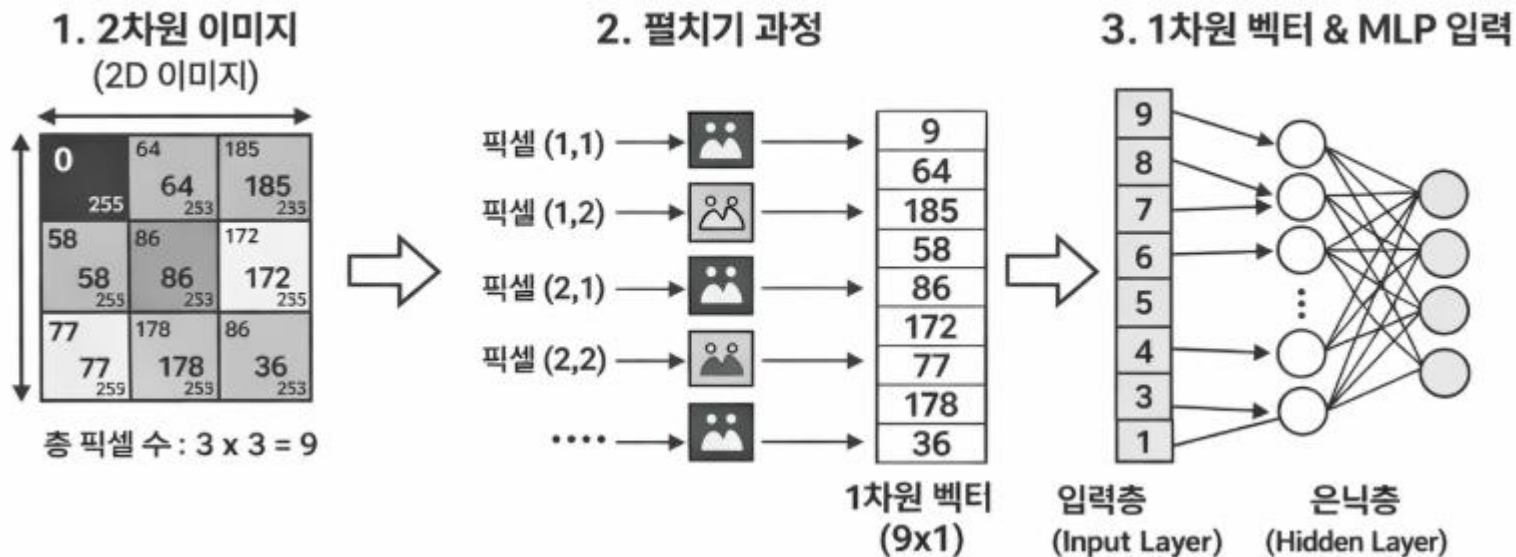


컴퓨터는

131	152	152	179	133	179
128	160	179	178	184	185
113	151	171	138	179	188

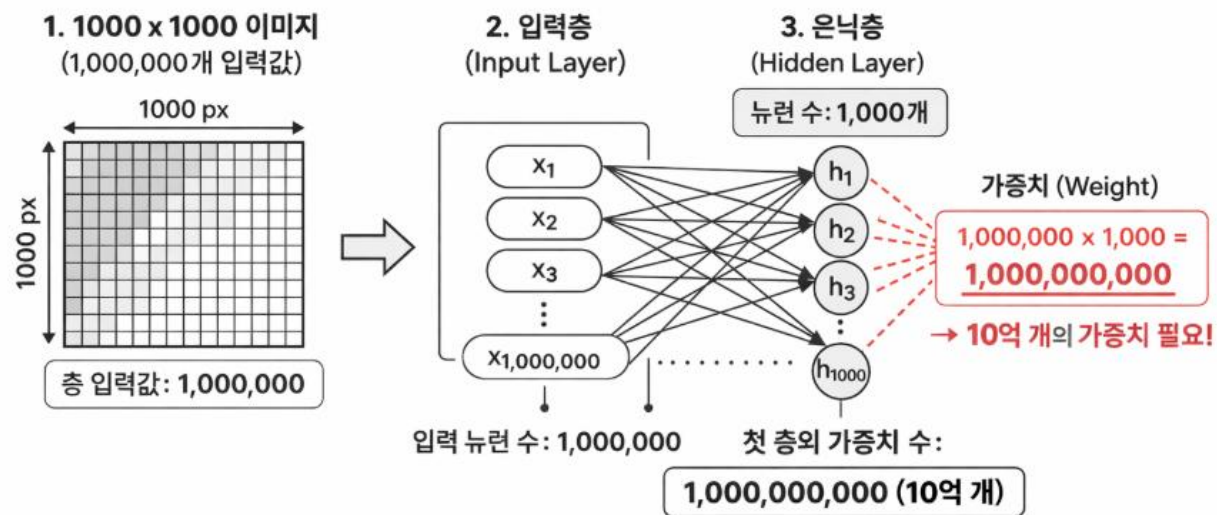
✓ MLP의 한계 (1/2)

- MLP는 데이터를 1차원 배열로 입력 받음
- 이미지를 MLP로 학습 시, 2차원 배열 형태의 숫자로 변환된 이미지를 1차원 벡터로 펼치는 Flatten 과정을 거쳐야 함
- 이로 인해, 인접한 픽셀끼리 밀접한 관계(선, 면, 형상)이 모두 사라져서 특징을 읽기 어려워짐



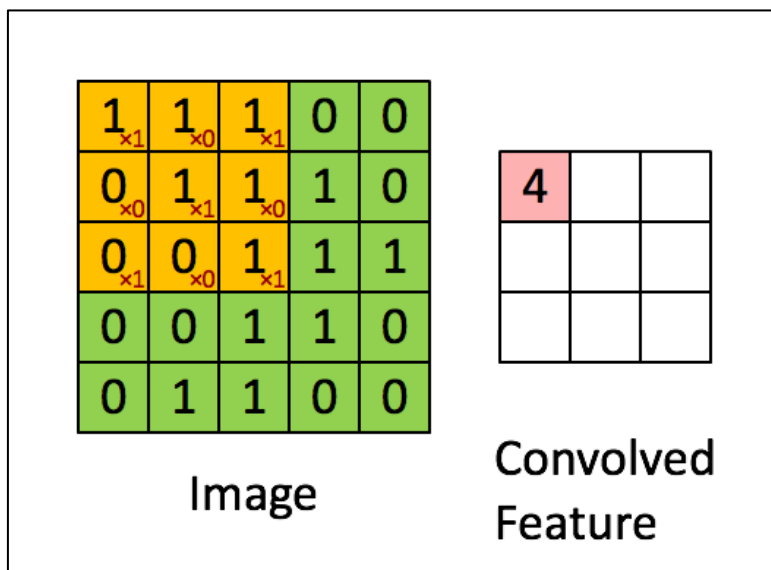
✓ MLP의 한계 (2/2)

- MLP는 이전 층의 모든 뉴런이 다음 층의 모든 뉴런과 연결되는 완전 연결 구조임
- 그래서 고해상도의 이미지를 입력하게 되면 많은 가중치로 인해 학습 속도를 느리게 하고, 과적합 위험이 있음
- 예를 들어,
 - 1000 * 1000 해상도의 이미지는 100만개의 입력값을 가짐
 - 은닉층 뉴런이 1,000 개라면 총 100만 * 1,000 으로 10억개의 가중치가 필요함



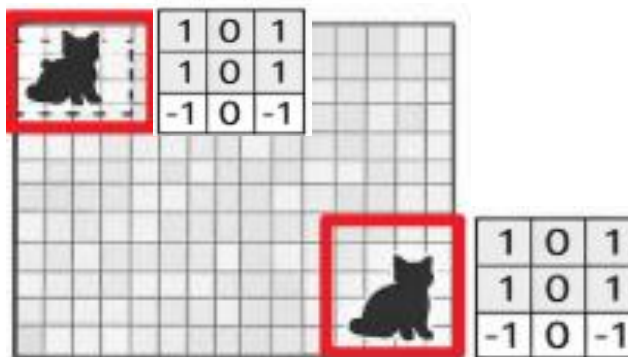
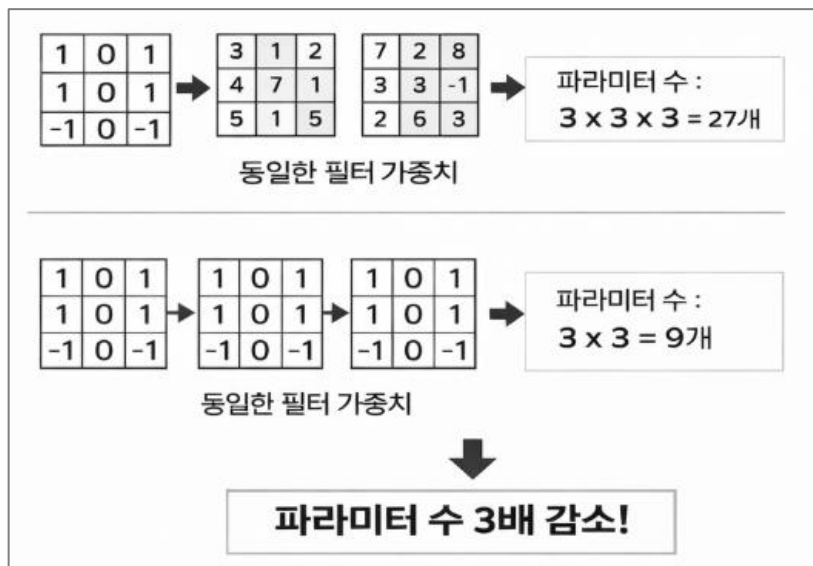
✓ CNN의 등장 (1/3)

- MLP의 한계
 - 2차원 배열 형태의 숫자로 변환된 이미지를 1차원 벡터로 펼치는 Flatten 과정
- CNN의 극복 방안
 - 이미지를 펼치는 대신, 작은 필터(Filter/Kernel)를 사용해 **이미지를 훑으며 특징을 찾아냄**



✓ CNN의 등장 (2/3)

- MLP의 한계
 - MLP는 이전 층의 모든 뉴런이 다음 층의 모든 뉴런과 연결되는 완전 연결 구조
- CNN의 극복 방안
 - 이미지 전체에 **동일한 필터 가중치**를 적용해 파라미터 수를 줄임
 - 동일한 가중치의 필터가 이미지 전체를 훑기 때문에, 특징이 어디에 있든 똑같이 추출할 수 있음



✓ CNN의 등장 (3/3)

- CNN은 이미지의 구조를 유지하면서 효율적으로 학습하기 위해서 등장
- 이미지를 학습한다는건 ?
 - 단순히 픽셀값을 외우는 것이 아님
 - 데이터를 요약하여 대상을 설명할 수 있는 특징을 추출하는 것이 목표
 - 저수준(선, 점, 색상) → 고수준(눈, 코, 창문과 같은 형태) → 최종 (특징을 조합해 고유의 특징맵을 만드는 것)
 - 우리는 어떤 사진이 들어와도 그 대상의 특징을 가장 잘 끄집어낼 수 있는 “필터”를 찾아내는 것이 목표!

The image features a central white hexagon with a thin white border, set against a background of overlapping blue triangles in various shades. Several small, bright white star-like lights are scattered across the blue background. The letters "CNN" are centered within the white hexagon in a bold, black, sans-serif font.

CNN

✓ CNN의 핵심 아이디어

- 작은 필터로 국소 영역을 본다
- 같은 필터를 이미지 전체에 반복 적용한다.
- 특징을 단계적으로 추출한다.

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

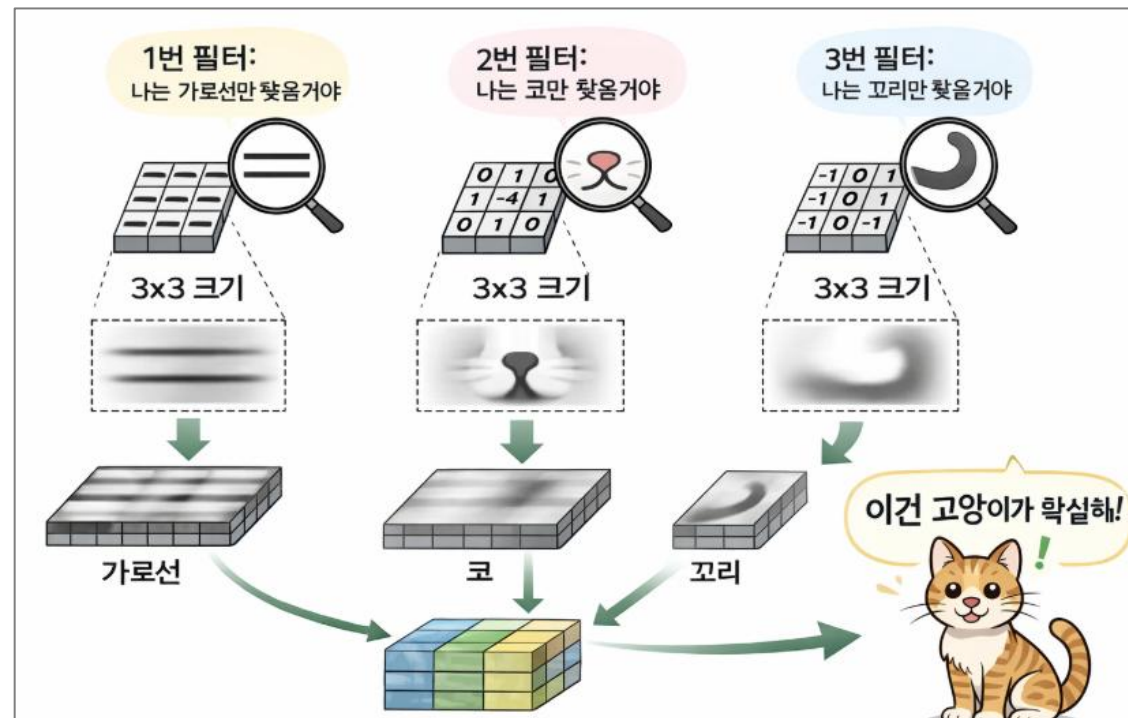
Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

✓ 커널/필터 (1/2)

- 이미지의 특징(세로선, 색상 변화 등)을 찾아내기 위한 작은 행렬
- 각 셀은 가중치로 채워져 있음
- 필터마다 찾고자 하는 특징이 다름
- 예를 들어)
 - 1번 필터: 가로선만 찾을거야,
 - 2번 필터: 코만 찾을거야
 - 3번 필터: 꼬리만 찾을거야
 - 이런 필터들이 층이 중첩되면서 정보들이 조합되고
“이건 고양이가 확실해”라고 판단하게 되는 것

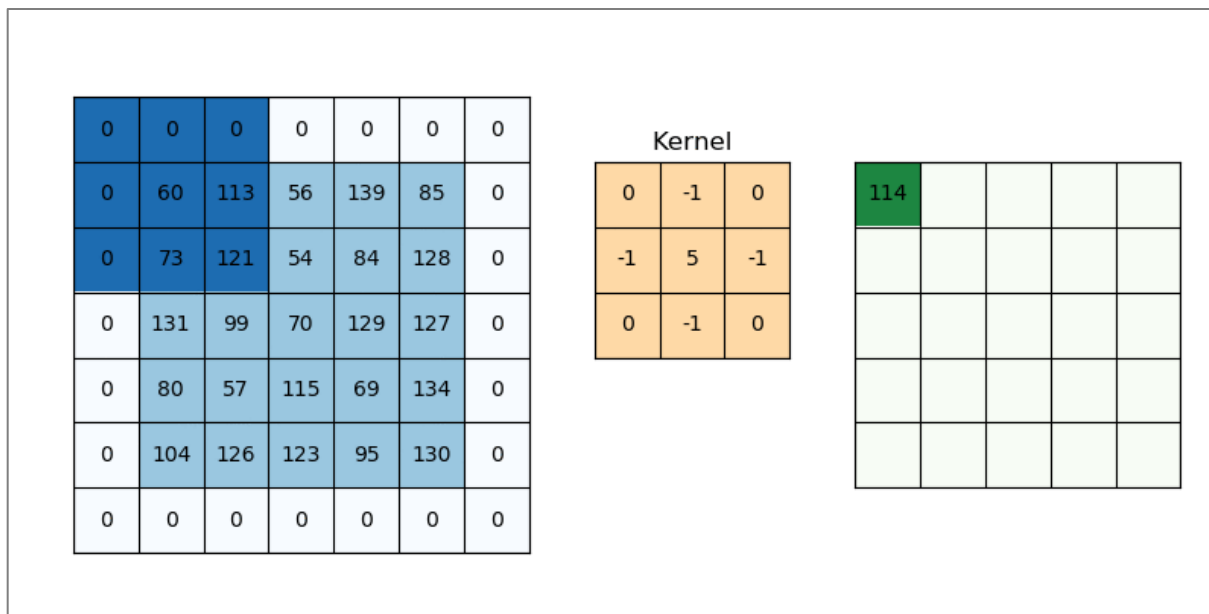


✓ 커널/필터 (2/2)

- 정확히는 커널과 필터는 다른 용어
 - 커널:(Kemel): 가장 작은 단위의 2차원 가중치 행렬이며, 입력 데이터의 “단일 채널”에 적용되어 특징을 추출
 - 필터:(Filter) : 여러 개의 커널이 모여 있는 3차원 텐서 묶음이며, 입력 데이터의 “모든 채널”을 동시에 처리
 - RGB 컬러 이미지(3채널)을 처리하기 위해서는 빨강, 초록, 파랑을 각각 담당하는 3개의 커널을 가져야 함
 - 왜 혼용해서 사용하나요? 입문 자료에서는 설명을 단순화하기 위해 채널이 1개인 흑백 이미지를 예로 들기 때문
- CNN은 커널/필터가 이미지 위를 일정한 간격으로 겹쳐 지나가며 수학적 연산을 수행하여 특징을 추출!
- 이 수학적 연산이 **“합성곱 연산”**

✓ 합성곱(Convolution) 연산 (1/2)

- 필터를 이미지 위에 겹쳐 놓고 곱한 뒤 더해 하나의 값을 만드는 연산
- 필터가 원하는 특징이 어디에 있는지 찾는 과정
- 그리고 이러한 수학적 연산의 결과값이 모여서 새로운 2차원 배열이 생기는데 이것이 “특징맵”



합성곱 연산 과정

✓ 합성곱(Convolution) 연산 (2/2)

- PyTorch에서는 `nn.Conv2d`를 사용하여 2D 이미지에 대한 합성곱 계층을 정의
- `nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5)`의 의미:
 - `in_channels=3`: 입력 이미지의 채널 수 (RGB)
 - `out_channels=6`: 사용할 필터의 개수. 즉, 생성될 특징 맵의 개수
 - `kernel_size=5`: 필터의 크기 (5x5)
- 실습 코드에서는...
 - 첫 번째 합성곱 층에서는 3채널 이미지를 입력받아 6개의 특징 맵을,
두 번째 층에서는 6개의 특징 맵을 입력받아 16개의 더 복잡한 특징 맵을 추출

```
self.conv1 = nn.Conv2d(3, 6, 5)
self.conv2 = nn.Conv2d(6, 16, 5)
```

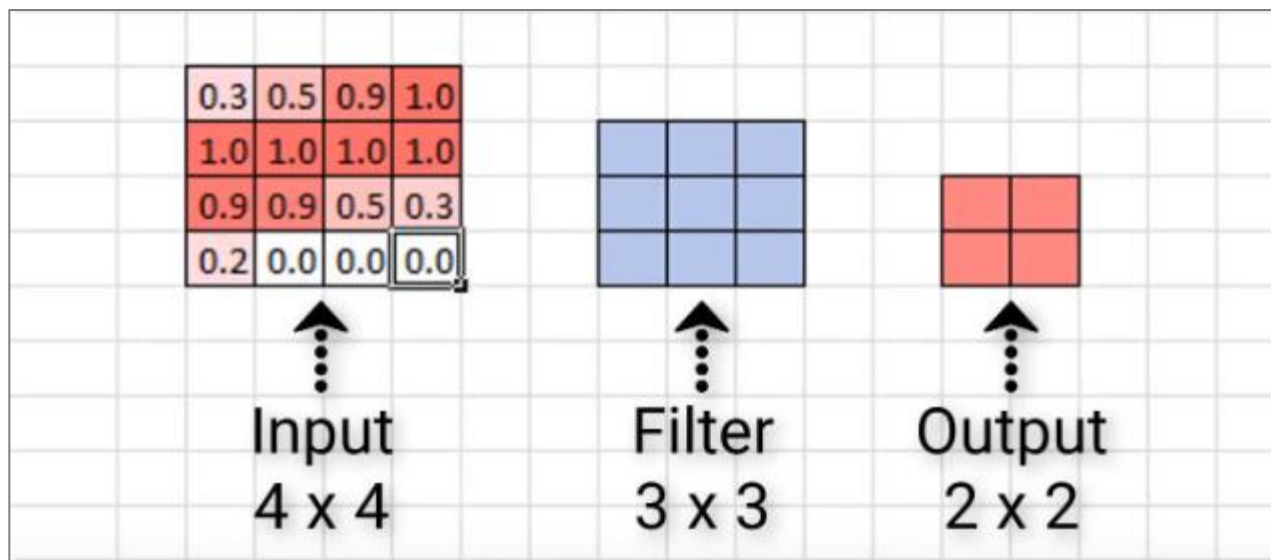
✓ 특징맵(Feature Map)

- 합성곱 연산의 결과값이 모여서 만들어진 2차원 배열
- 필터가 찾고자 하는 특징을 수치로 적어놓은 것이라고 이해하면 됨
- 높은 숫자(활성화): 필터가 찾고자 하는 특징이 해당 위치에서 강하게 나타날 때
- 낮은 숫자: 특징이 없거나 약한 경우 0에 가까운 값이 기록



✓ 패딩(Padding) (1/2)

- 합성곱 연산을 수행하기 전, 입력 데이터의 외곽에 특정 값(주로 0)을 채워 넣는 것을 의미
- 왜 패딩이 필요한가요?
 - 합성곱 연산을 거치면서 **특징맵의 크기가 줄어드는데** 이렇게 되면 층을 깊게 쌓을 수 없음
 - 패딩을 통해 입력 크기를 출력에서도 그대로 유지할 수 있게 하여, 신경망을 깊게 설계할 수 있도록 하기 위해서



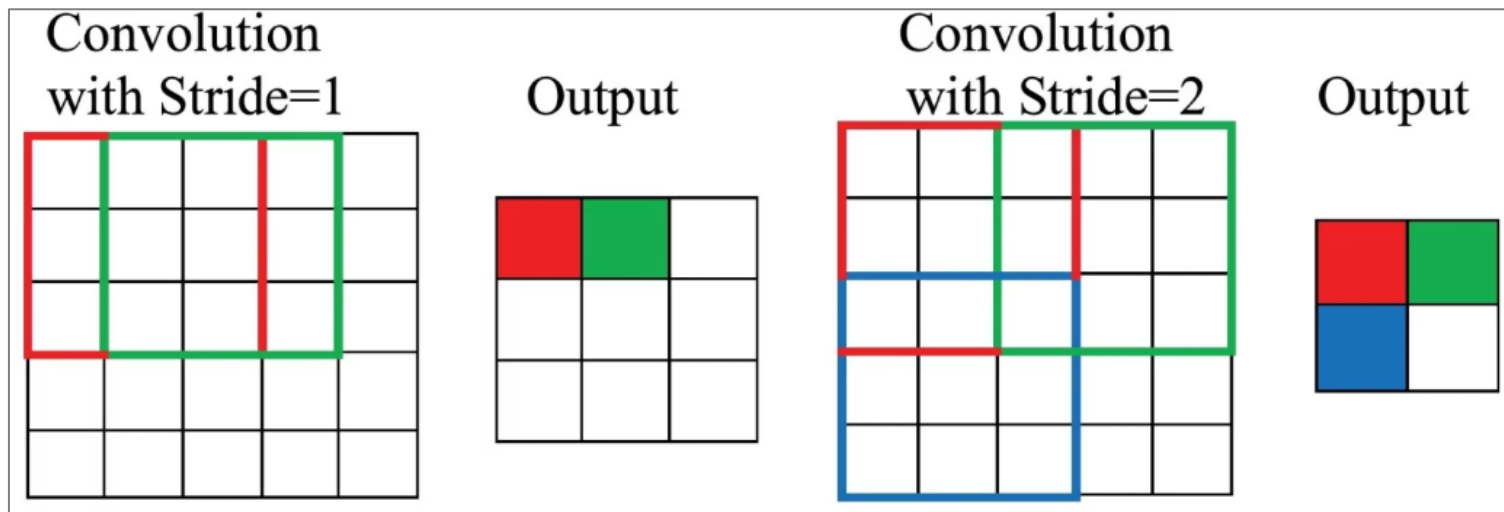
✓ 패딩(Padding) (2/2)

- 합성곱 연산을 수행하기 전, 입력 데이터의 외곽에 특정 값(주로 0)을 채워 넣는 것을 의미
- 왜 패딩이 필요한가요?
 - 이미지의 중심부 픽셀은 필터가 여러 번 훑으며 연산에 참여하지만, 외곽 픽셀은 연산에 참여하는 횟수가 적음
 - 패딩을 주면 외곽 데이터도 연산의 중심에 올 수 있게 되어 중요한 특징을 골고루 추출할 수 있음
 - 패딩이 없다면 1번만 연산에 참여했지만, 패딩으로 인해 여러 번 참여할 수 있게 됨



✔ 스트라이드(Stride) (1/2)

- 필터를 입력 데이터 위에서 이동시킬 때의 간격을 의미
- 보통 1의 값을 가지며, 필터를 한 픽셀씩 옆으로 옮기며 연산한다는 의미
- 스트라이드를 조절하는 이유1
 - 스트라이드 값이 커질수록 필터가 건너뛰는 칸수가 많아지므로, 특징맵의 크기가 급격히 작아짐
 - 이는 데이터를 압축하는 효과를 줌



✓ 스트라이드(Stride) (2/2)

- 스트라이드를 조절하는 이유 2
 - 필터가 연산하는 횟수 자체가 줄어들기 때문에 전체적인 연산량이 감소
- 스트라이드를 조절하는 이유 3
 - 큼직하게 건너뛰며 볼 때, 필터가 이미지의 더 넓은 영역을 한 번에 훑는 효과가 생김
 - 전체적인 구도를 파악하는 데 유리할 수 있음
- 가장 중요한 기능은 “데이터의 핵심적인 특징만 남겨서 압축”

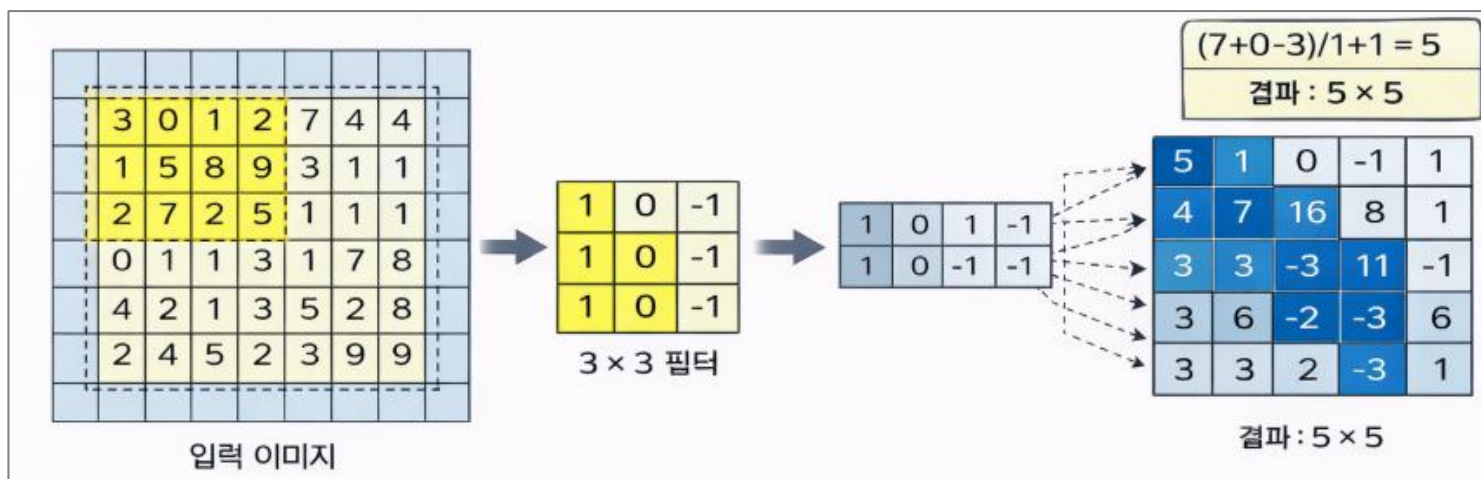
✓ 지금까지 배운 내용을 기반으로 특징맵을 계산해보자 (1/2)

- 입력 이미지: $7 * 7$
- 필터: $3 * 3$
- 패딩 :0
- 스트라이드:1
- 특징맵의 크기는??

✓ 지금까지 배운 내용을 기반으로 특징맵을 계산해보자 (2/2)

- 입력 이미지(I): 7×7
- 필터(F): 3×3
- 패딩(P): 0
- 스트라이드(S): 1
- 특징맵(O)의 크기는?? 5×5

$$O = \frac{I - F + 2P}{S} + 1$$



✓ 활성화 함수: ReLU

- 합성곱 연산이 끝나고 나온 특징맵에 대해서 활성화 함수(ReLU)에 통과시킴
- 활성화 함수에 통과시키는 이유
 - 합성곱 연산은 '선형 연산'이며, 아무리 계속해도 선형 함수일 뿐이라서 복잡한 이미지를 학습할 수 없음
 - 복잡한 패턴을 학습하려면 '비선형성'을 추가해야하고, 이를 위해 활성화 함수를 활용
- 현대 CNN에서는 거의 무조건 ReLU를 사용
 - ReLU: 입력값이 0보다 크면 그대로 내보내고, 0보다 작으면 0으로 만들어버림
- 기울기 소실 문제 해결 + 계산의 단순함 + 중요하지 않은 정보를 0으로 처리

5	0	0	8	1
10	7	16	8	1
0	3	-3	11	-1
3	6	-2	-3	6
3	2	3	-3	3

입력 특징맵

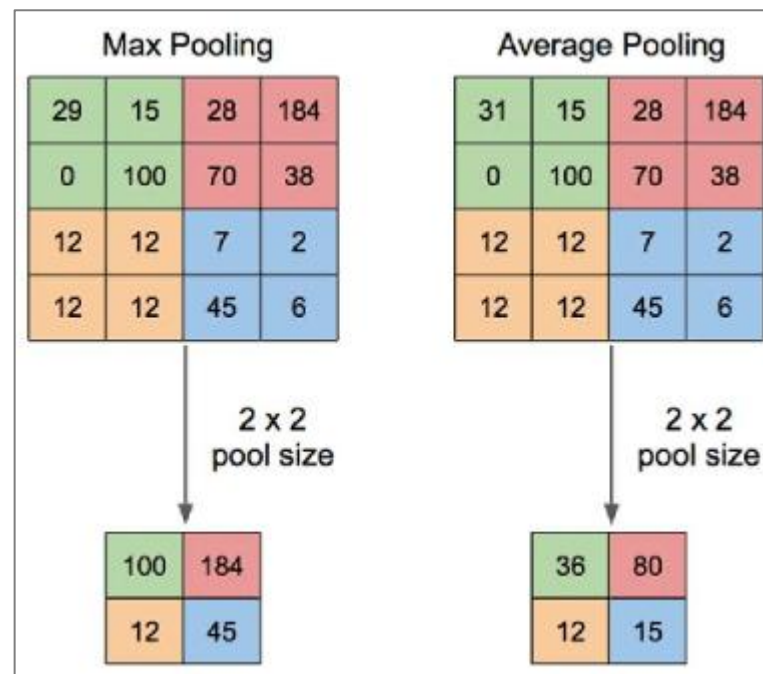
→

5	0	0	8	1
0	7	16	8	1
0	3	0	11	0
3	6	0	0	0
3	2	3	0	0

ReLU 적용 후

✓ 폴링(Pooling) (1/2)

- 특징맵의 크기를 줄여서 계산량을 줄이고, 주요 특징을 추출(압축)하는 단계
- 합성곱 연산과는 달리 가중치가 전혀 없고, 정해진 규칙에 따라 계산만 진행
- 폴링의 종류 (계산하는 방식에 따라 나뉨)
 - Max Pooling (많이 사용)
 - 설정한 영역에서 가장 큰 값만 뽑아냄
 - 해당 영역에서 특징이 가장 강하게 나타난 부분만 대표로 활용
 - 노이즈를 제거하고 가장 선명한 특징만 강조하는 효과가 있음
 - Average Pooling
 - 설정한 영역 안의 값들의 평균을 내어 전달
 - 전체적인 분위기를 전달하는 용도로 활용
 - 핵심 특징이 희석될 수 있어서 마지막 layer 외에는 잘 사용하지 않음



✓ 폴링을 하는 이유

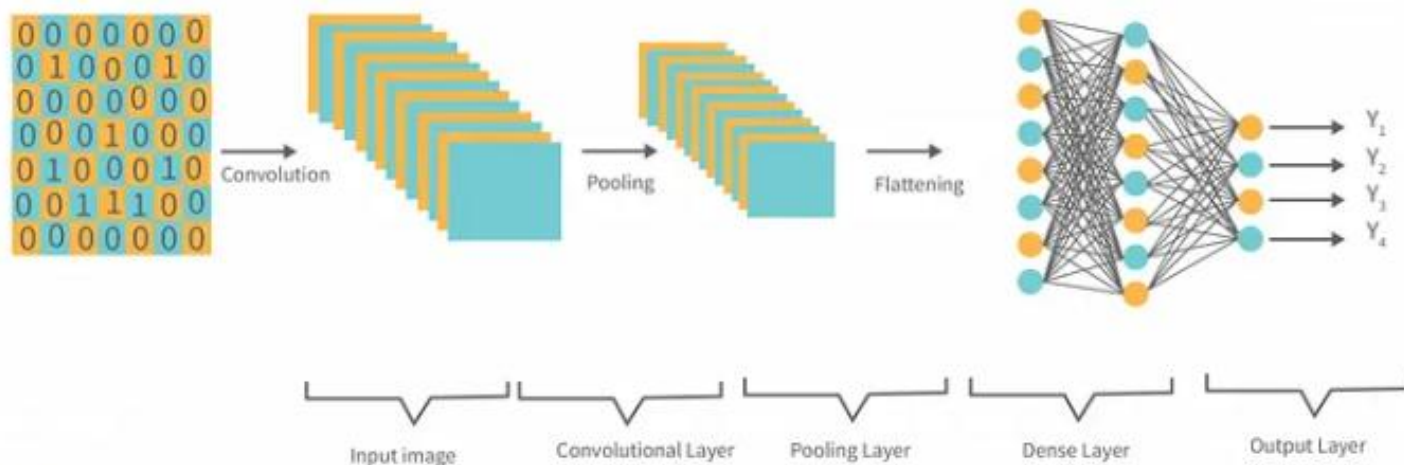
1. 이미지 크기를 줄여서 학습 연산의 부담을 줄이기 위해서
2. 너무 세세한 정보까지 학습하지 않게 데이터를 단순화시키기 위해서 (과적합 방지)
3. 합성곱은 채널을 합치지만, 폴링은 각 채널별로 따로 작동하여 채널 수가 변하지 않음
 - 합성곱: 빨강, 초록, 파랑 물감을 섞어서 “새로운 색”을 만들어내는 과정
 - 폴링: 빨간 종이, 초록 종이, 파랑 종이를 각각 똑같은 크기로 접어서 다시 쌓아두는 과정 (특성 보존)
4. 대상이 이미지 안에서 살짝 움직이더라도, 폴링 영역 안에서 결과는 동일할 수 있음 (위치에 덜 민감함)

✓ 지금까지 배운 내용 정리

- CNN은 추출한 특징을 여러 겹으로 쌓는 과정
 1. 합성곱(Convolution) + (Padding/Stride) : 필터로 이미지의 특징을 찾아내서 특징맵을 생성
 2. ReLU: 찾아낸 특징 중 의미있는 데이터만 남김 (음수는 0으로)
 3. Pooling: 데이터를 압축한다.
 4. 위 과정을 반복하여 점/선 → 눈/코 → 얼굴 과 같이 고차원 특징을 완성한다.

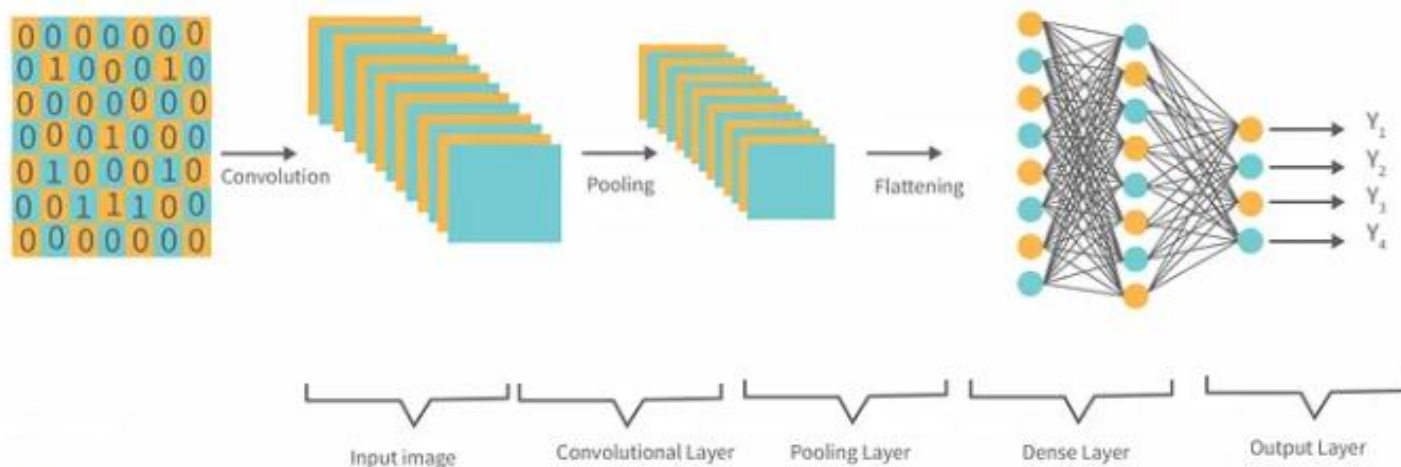
✓ Flatten (1/2)

- 지금까지는 이미지의 '특징'을 추출했고, 이 정보는 가로, 세로, 채널이 있는 3차원 정육면체(Tensor)의 형태
- 하지만 우리가 분류 모델로 사용하는 일반적인 MLP은 1차원 벡터만 입력으로 받을 수 있음
- 3차원의 특징맵을 한 줄(1차원)로 길게 늘어뜨리는 작업
- Flatten을 진행하면 공간 정보가 사라지나요?



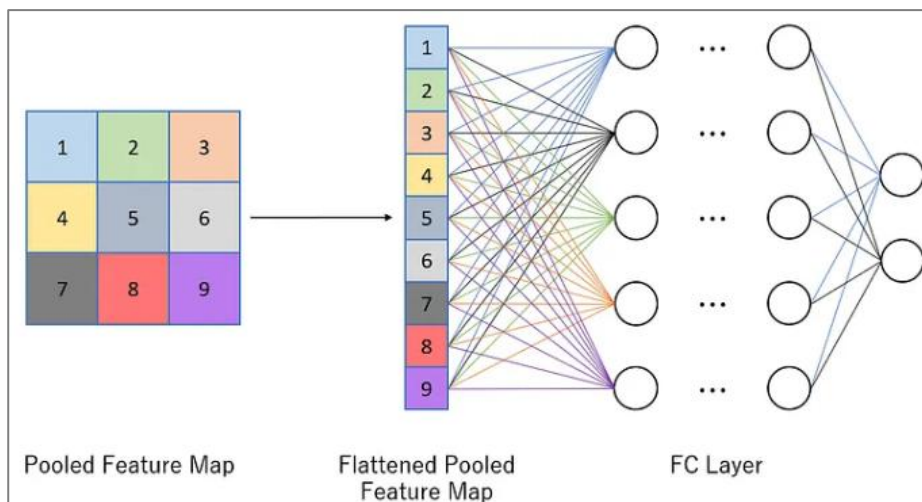
✓ Flatten (2/2)

- 지금까지는 이미지의 '특징'을 추출했고, 이 정보는 가로, 세로, 채널이 있는 3차원 정육면체(Tensor)의 형태
- 하지만 우리가 분류 모델로 사용하는 일반적인 MLP은 1차원 벡터만 입력으로 받을 수 있음
- 3차원의 특징맵을 한 줄(1차원)로 길게 늘어뜨리는 작업
- Flatten을 진행하면 공간 정보가 사라지나요? **아니요!** 하지만 합성곱/풀링에서 특징을 다 뽑아냈기 때문에 괜찮!



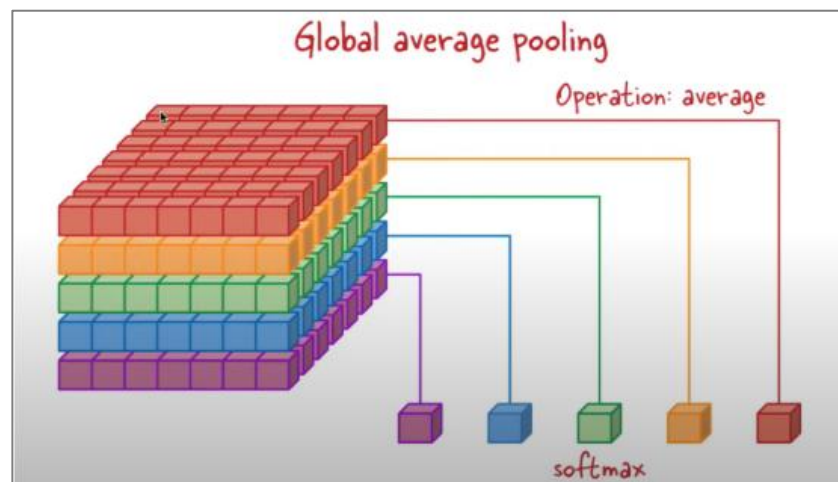
✓ Fully Connected Layer (FC Layer)

- 펼쳐진 벡터를 입력으로 받아 특징들을 하나로 조합하여 이미지가 어떤 클래스에 속하는 지 분류하는 역할
- 다중 클래스 분류를 위해 “소프트맥스 활성화 함수”를 활용함
- 4장에서 학습한 MLP의 동작과정과 동일하게 진행되어 분류 작업을 진행함
- 하지만! 여전히 파라미터(가중치) 수가 너무 많아 메모리를 많이 차지하고 과적합 위험이 있음
 - 그래서 최근에는 Flatten 대신에 **GAP(Global Average Pooling)** 기법을 사용



✓ GAP(Global Average Pooling)

- 특징맵 전체를 대상으로 평균을 내는 풀링 기법
- Flatten으로 펼쳐진 수천 개의 노드와 다음 층을 연결하기 위해서 수많은 가중치를 줄이기 위해 사용
- 작동 방식
 - 각 채널의 가로 * 세로 크기에 상관없이, 한 채널의 모든 원소값을 평균내어 하나의 숫자로 만듦
 - ex) $7 * 7 * 512$ 크기의 데이터가 있다면, GAP을 거치면 바로 $1 * 1 * 512$ 가 됨
 - $\Rightarrow 1 * 1$ 에는 해당 채널 값들의 평균이 들어감
- 각 채널별로 평균을 따로 구하는 것
 - 채널은 각각 서로 다른 특징을 의미
 - 이는 각 특징들의 요약을 구하는 것을 의미



✓ CNN 핵심 아키텍처 요약

- 1. 특징 추출 (Feature Extraction)
 - Conv (합성곱): 필터로 이미지의 특징(선, 면, 패턴)을 찾아냄 (Padding/Stride로 크기 제어)
 - ReLU (활성화): 음수 데이터를 0으로 처리하여 '의미 있는 특징'만 활성화
 - Pooling (압축): 특징의 위치에 상관없이 핵심 정보만 남기고 데이터 크기 축소
- 2. 데이터 브릿지 (Bridge)
 - Flatten: 3D 특징맵을 1차원 벡터로 길게 펼치기 (전통적 방식)
 - GAP (Global Average Pooling): 채널별 평균값만 뽑아 핵심 요약 (과적합 방지)
- 3. 최종 분류 (Classification)
 - FC Layer (전결합층): 추출된 특징들을 가중치로 조합하여 클래스별 점수 계산 (MLP 구조)
 - Softmax (최종 출력): 점수를 0~1 사이의 확률로 변환 (모든 합 = 100%)



다양한 CNN 모델

✓ 다양한 CNN 모델과 현재의 이미지 처리 방식 (1/2)

- AlexNet (2012): 딥러닝 패러다임의 전환
 - ReLU 활성화 함수를 사용하여 기울기 소실 문제를 완화하고 학습 속도를 높임
 - Dropout을 통해 과적합을 방지하며, GPU 연산을 본격적으로 도입하여 대규모 데이터를 처리
- VGGNet (2014): 구조의 단순화와 깊이의 증명
 - 모든 층에서 3*3 크기의 작은 커널만을 사용해서, 파라미터 수를 줄이고 깊게 쌓아 성능을 향상
 - 아키텍처 설계의 표준화를 제시
- ResNet (2015): 잔차 학습을 통한 층의 수직적 확장
 - 네트워크가 일정 수준 이상 깊어지면 오히려 성능이 떨어지는 '망의 퇴화' 문제를 해결한 모델
 - 입력 데이터를 출력에 직접 더해주는 방식을 통해 기울기가 소실되지 않고 신경망의 하위 층까지 잘 전달되도록 설계
=> “잔차 학습(Residual Learning)”

✓ 다양한 CNN 모델과 현재의 이미지 처리 방식 (2/2)

- ViT (Vision Transformer): 새로운 패러다임의 제시
 - 자연어 처리에서 성공을 거둔 Transformer 구조를 이미지 처리에 도입한 모델
 - 이미지를 작은 패치 단위로 분할하여 문장 내 단어처럼 취급하고, Self-Attention 매커니즘을 적용함
 - CNN의 본질적인 제약인 “지역적 연산”에서 벗어나 이미지 전체의 문맥을 한 번에 평가
 - 지역적 연산: 이미지를 처리할 때, 전체 이미지가 아닌 주변의 인접한 픽셀들만 고려하여 연산하는 방식
 - 거대 데이터셋 학습 시 CNN의 성능 한계를 뛰어넘으며, 현재 다양한 기술이 개발되고 있지만 근본으로 자리잡은 기술